

2 - GoMuseum开发指南-MVP核心功能

开发概述

本文档涵盖GoMuseum项目的MVP核心功能开发，包含Step 1到Step 5的完整实施指南。每个Step都采用TDD开发模式，确保代码质量和功能稳定性。

总体目标: 实现可用的博物馆导览APP，支持拍照识别、AI讲解、支付功能和基础UI界面。

Step 1 - 图像识别功能开发

概览

目标

- 实现拍照识别展品功能
- 集成OpenAI GPT-4V API
- 建立识别结果缓存机制
- 实现5秒内响应目标

Agent角色

优先调用相关agents，如没有匹配的agent你需要扮演以下角色：

- AI集成工程师:** 负责OpenAI API集成和识别逻辑
- Flutter开发工程师:** 负责相机功能和图像处理
- 后端工程师:** 负责识别服务API开发
- 测试工程师:** 负责TDD测试用例设计

工作量估算

- 预估Token消耗: 60K tokens

- 预估交互次数: 8-10次
- 预估开发时间: 6-8小时
- 复杂度等级: 中-高

Claude Code完整指令

markdown

请首先阅读并理解以下GoMuseum完整架构文档章节：

GoMuseum架构背景 - 图像识别模块

3.1 拍照识别模块设计

3.1.1 识别流程

用户拍照 -> 本地缓存检查 -> AI识别 -> 结果融合 -> 返回结果 -> 更新缓存

3.1.2 技术实现

- **主要AI模型**: OpenAI GPT-4V
- **响应时间要求**: P95 < 5秒
- **缓存策略**: 本地SQLite + Redis云端缓存
- **降级方案**: GPT-4V -> Claude Vision -> 本地模型 -> 手动搜索

4.2 AI识别优化

- **提示词工程**: 专业的艺术史专家角色设定
- **多级降级策略**: 3秒超时的多层fallback
- **结果格式**: 标准JSON格式返回

5.1 数据库设计

- **用户表**: 存储用户信息和权益
- **展品表**: 存储展品信息和向量嵌入
- **识别缓存表**: 存储识别结果缓存
- **用户足迹表**: 记录用户识别历史

6.1 并发控制

- **GPU并发限制**: 10个并发请求
- **请求队列**: 1000个任务队列
- **超时处理**: 5秒总超时

Step 1 - 图像识别功能开发任务

TDD开发模式

严格按照红灯-绿灯-重构的TDD流程：

1. **红灯阶段**: 先写失败的测试用例
2. **绿灯阶段**: 实现最小可行代码使测试通过
3. **重构阶段**: 优化代码结构和性能

角色设定

优先调用相关agents，如没有匹配的agent你需要扮演以下角色：

1. **AI集成工程师**: 负责OpenAI API集成和识别逻辑
2. **Flutter开发工程师**: 负责相机功能和图像处理
3. **后端工程师**: 负责识别服务API开发
4. **测试工程师**: 负责TDD测试用例设计

具体开发任务

第一轮TDD - 基础识别功能

红灯阶段 - 编写失败测试:

1. 测试相机拍照功能
2. 测试图像上传到后端
3. 测试OpenAI API调用
4. 测试识别结果解析
5. 测试错误处理机制

绿灯阶段 - 实现核心功能:

1. Flutter相机集成 (`image_picker` + `camera`)
2. 图像处理和压缩
3. 后端FastAPI识别接口
4. OpenAI GPT-4V集成
5. 结果JSON解析

重构阶段 - 优化代码:

1. 提取可复用组件
2. 优化图像处理性能
3. 添加错误处理
4. 完善日志记录

第二轮TDD - 缓存机制

红灯阶段 - 缓存测试:

1. 测试本地缓存存储

2. 测试缓存命中逻辑
3. 测试缓存过期机制
4. 测试缓存清理策略

****绿灯阶段 - 实现缓存**:**

1. Drift本地数据库配置
2. Redis云端缓存集成
3. 图像特征哈希算法
4. 缓存策略实现

第三轮TDD - 性能优化

****红灯阶段 - 性能测试**:**

1. 测试5秒响应时间要求
2. 测试并发识别能力
3. 测试降级方案
4. 测试离线模式

****绿灯阶段 - 性能实现**:**

1. 请求超时控制
2. 并发限制机制
3. 多级降级策略
4. 离线缓存回退

关键实现文件

Flutter端文件

```
lib/features/recognition/  
├── data/  
│   ├── datasources/  
│   │   ├── recognition_local_datasource.dart  
│   │   └── recognition_remote_datasource.dart  
│   └── models/  
│       └── recognition_result_model.dart  
└── repositories/  
    └── recognition_repository_impl.dart
```

```
└── domain/
|   └── entities/
|       └── recognition_result.dart
|   └── repositories/
|       └── recognition_repository.dart
|   └── usecases/
|       ├── recognize_artwork.dart
|       └── get_cached_result.dart
└── presentation/
    ├── providers/
    │   └── recognition_provider.dart
    ├── pages/
    │   └── camera_page.dart
    └── widgets/
        ├── camera_widget.dart
        └── recognition_result_widget.dart
```

后端文件

```
backend/app/
└── api/v1/
    └── recognition.py
└── services/
    ├── ai_service.py
    ├── cache_service.py
    └── image_service.py
└── models/
    ├── recognition.py
    └── artwork.py
└── schemas/
```

- └── recognition_request.py
- └── recognition_response.py

测试文件

test/features/recognition/

- └── data/
- | └── datasources/
- | └── repositories/
- └── domain/
- | └── usecases/
- └── presentation/
- └── providers/

backend/tests/

- └── unit/
- | └── test_ai_service.py
- | └── test_cache_service.py
- | └── test_recognition_api.py
- └── integration/
- | └── test_recognition_flow.py
- └── e2e/
- └── test_recognition_e2e.py

技术要求

1. **Flutter**: 使用Clean Architecture + Riverpod
2. **后端**: 使用FastAPI + SQLAlchemy + Alembic
3. **AI集成**: OpenAI GPT-4V API with fallback
4. **缓存**: Drift (本地) + Redis (云端)
5. **测试**: 完整的单元测试和集成测试
6. **错误处理**: 完善的异常处理和用户提示
7. **日志**: 详细的操作日志记录

验收标准

1. 识别功能正常工作，P95响应时间 < 5秒
2. 缓存机制有效，命中率 > 60%
3. 错误处理完善，用户体验良好
4. 测试覆盖率: Flutter > 80%, Python > 85%
5. 通过所有TDD测试用例
6. 代码质量通过静态分析检查

请严格按照TDD模式开发，每个阶段都要先写测试再实现功能。

总结

预期输出文件 (28个)

yaml

Flutter代码文件 (12个):

- recognition相关的data/domain/presentation层完整实现
- 相机功能和图像处理组件
- 状态管理和错误处理

Python后端文件 (8个):

- FastAPI识别接口
- AI服务集成
- 缓存服务
- 数据模型和Schema

测试文件 (8个):

- Flutter单元测试和Widget测试
- Python单元测试和集成测试
- E2E端到端测试

测试覆盖率目标

- Flutter单元测试: > 80%
- Flutter Widget测试: > 75%
- Python单元测试: > 85%
- 集成测试覆盖: > 80%
- 关键路径: 100%覆盖

验收

自动化验收脚本

```
bash
```

```
#!/bin/bash
```

```
# step1-acceptance.sh
```

```
echo "Step 1 - 图像识别功能验收测试"
```

```
# Flutter测试
```

```
cd frontend/gomuseum_app
```

```
echo "执行Flutter测试..."
```

```
flutter test --coverage
```

```
if [ $? -eq 0 ]; then
```

```
    echo "✅ Flutter测试通过"
```

```
else
```

```
    echo "❌ Flutter测试失败"
```

```
    exit 1
```

```
fi
```

```
# 检查覆盖率
```

```
flutter pub global activate coverage
```

```
flutter pub global run coverage:format_coverage --lcov --in=coverage --out=coverage/lcov.info
```

```
# Python后端测试
```

```
cd ../../backend
```

```
echo "执行Python测试..."
```

```
pytest tests/ -v --cov=app --cov-report=term-missing --cov-fail-under=85
```

```
if [ $? -eq 0 ]; then
```

```
    echo "✅ Python测试通过"
```

```
else
```

```
    echo "❌ Python测试失败"
```

```
    exit 1
```

```
fi
```

```
# 功能验收测试
```

```
echo "执行功能验收..."
```

```
python scripts/test_recognition_api.py
```

```
if [ $? -eq 0 ]; then
```

```
    echo "✅ 识别功能正常"
```

```
else
```

```
echo "❌ 识别功能异常"
exit 1
fi

echo "🎉 Step 1验收完成"
```

手工验收清单

- ☐ 打开相机拍照功能正常
- ☐ 识别请求在5秒内返回结果
- ☐ 识别结果准确率达到预期
- ☐ 缓存机制工作正常，二次识别更快
- ☐ 网络异常时有适当的错误提示
- ☐ 识别历史记录正确保存
- ☐ 测试覆盖率达到目标要求

版本管理和CI/CD

Git工作流

```
bash
```

创建功能分支

```
git checkout -b feature/recognition/core-implementation
```

开发完成后提交

```
git add .
```

```
git commit -m "feat(recognition): implement core image recognition functionality"
```

- Add camera capture and image processing
- Integrate OpenAI GPT-4V API with fallback
- Implement local and cloud caching system
- Add comprehensive error handling
- Achieve 82% test coverage

TDD implemented with:

- Red: 15 failing tests initially
- Green: All tests passing after implementation
- Refactor: Code optimization and cleanup

Closes #2"

推送并创建PR

```
git push origin feature/recognition/core-implementation
```

CI/CD检查

-  Flutter analyze无错误
-  Flutter test全部通过，覆盖率>80%
-  Python pytest全部通过，覆盖率>85%
-  API集成测试通过
-  安全扫描无高危漏洞
-  性能测试满足5秒响应要求

Step 2 - AI讲解生成功能

概览

目标

- 实现基于识别结果的AI讲解生成
- 支持多语言内容生成
- 实现流式内容返回
- 集成TTS语音合成

Agent角色

优先调用相关agents，如没有匹配的agent你需要扮演以下角色：

1. **AI内容工程师:** 负责讲解内容生成和优化
2. **多语言工程师:** 负责国际化和本地化实现
3. **音频工程师:** 负责TTS集成和音频处理
4. **UX工程师:** 负责内容展示和用户体验

工作量估算

- 预估Token消耗: 50K tokens
- 预估交互次数: 6-8次
- 预估开发时间: 5-7小时
- 复杂度等级: 中

Claude Code完整指令

markdown

请首先阅读并理解以下GoMuseum完整架构文档章节：

GoMuseum架构背景 - AI讲解生成模块

3.2 AI讲解生成

内容生成策略

- **渐进式生成**: 基础信息立即返回，详细内容流式生成
- **多语言支持**: EN/FR/DE/ES/IT/中文
- **个性化偏好**: 根据用户设置调整讲解风格
- **TTS集成**: 每种语言3-5种音色可选

提示词工程

专业艺术史专家角色，输出结构化JSON格式，包含：

- artwork_name: 展品名称
- artist: 艺术家
- period: 时期/年代
- description: 详细描述
- cultural_context: 文化背景
- interesting_facts: 有趣知识点

性能要求

- 基础信息响应: < 1秒
- 完整内容生成: < 3秒
- TTS音频生成: < 2秒
- 支持内容缓存和预加载

Step 2 - AI讲解生成功能开发任务

TDD开发模式

严格按照红灯-绿灯-重构的TDD流程开发。

角色设定

优先调用相关agents，如没有匹配的agent你需要扮演以下角色：

1. **AI内容工程师**: 负责讲解内容生成和优化
2. **多语言工程师**: 负责国际化和本地化实现

- 3. ****音频工程师****: 负责TTS集成和音频处理
- 4. ****UX工程师****: 负责内容展示和用户体验

具体开发任务

第一轮TDD - 基础内容生成

****红灯阶段****:

- 1. 测试内容生成API调用
- 2. 测试多语言内容请求
- 3. 测试结果格式化
- 4. 测试内容缓存机制

****绿灯阶段****:

- 1. 实现GPT-4内容生成服务
- 2. 实现多语言prompt模板
- 3. 实现内容结构化处理
- 4. 实现Redis内容缓存

****重构阶段****:

- 1. 优化prompt工程
- 2. 完善错误处理
- 3. 添加内容质量检查

第二轮TDD - TTS语音合成

****红灯阶段****:

- 1. 测试TTS服务调用
- 2. 测试多语言语音合成
- 3. 测试音频文件处理
- 4. 测试音频缓存机制

****绿灯阶段****:

- 1. 集成OpenAI TTS API
- 2. 实现多语言音色选择
- 3. 实现音频文件上传和CDN
- 4. 实现客户端音频播放

第三轮TDD - 流式内容展示

****红灯阶段**:**

1. 测试流式内容接收
2. 测试渐进式UI更新
3. 测试音频播放控制
4. 测试离线内容访问

****绿灯阶段**:**

1. 实现Server-Sent Events
2. 实现Flutter流式UI更新
3. 实现音频播放控制器
4. 实现离线内容存储

关键实现文件

Flutter端

lib/features/content/

```
|—— data/
|   |—— datasources/
|   |   |—— content_remote_datasource.dart
|   |   |—— audio_local_datasource.dart
|   |—— models/
|   |   |—— explanation_model.dart
|   |   |—— audio_content_model.dart
|   |—— repositories/
|   |   |—— content_repository_impl.dart
|—— domain/
|   |—— entities/
|   |   |—— explanation.dart
|   |   |—— audio_content.dart
|   |—— repositories/
|   |   |—— content_repository.dart
|   |—— usecases/
|   |   |—— generate_explanation.dart
```

```
|   |—— get_audio_explanation.dart
|   |—— cache_content.dart
|   |—— presentation/
|   |—— providers/
|   |—— content_provider.dart
|   |—— audio_provider.dart
|   |—— pages/
|   |—— explanation_page.dart
|   |—— widgets/
|   |—— explanation_card.dart
|   |—— audio_player_widget.dart
|   |—— language_selector.dart
```

后端文件

```
backend/app/
|—— api/v1/
|   |—— content.py
|   |—— services/
|   |—— content_generation_service.py
|   |—— tts_service.py
|   |—— translation_service.py
|   |—— content_cache_service.py
|   |—— models/
|   |—— content.py
|   |—— audio.py
|   |—— schemas/
|   |—— content_request.py
|   |—— content_response.py
|   |—— audio_response.py
```

技术要求

1. **内容生成**: OpenAI GPT-4 API with structured prompts
2. **TTS服务**: OpenAI TTS API + 音频CDN存储
3. **多语言**: Flask-Babel + 本地化资源文件
4. **流式传输**: Server-Sent Events + Flutter Stream
5. **音频播放**: just_audio + 缓存管理
6. **离线支持**: 预下载热门内容

验收标准

1. 内容生成功能正常，响应时间达标
2. 多语言切换正常工作
3. TTS语音合成质量良好
4. 流式内容展示体验流畅
5. 测试覆盖率达标
6. 离线模式基础可用

请严格按照TDD模式，确保每个功能都有对应的测试用例。

总结

预期输出文件 (22个)

yaml

Flutter代码文件 (10个):

- 内容生成相关的完整Clean Architecture实现
- 多语言支持和音频播放组件
- 流式内容展示UI组件

Python后端文件 (7个):

- 内容生成API和TTS服务
- 多语言处理和缓存服务
- 内容质量控制模块

测试文件 (5个):

- 内容生成测试
- TTS功能测试
- 多语言测试

测试覆盖率目标

- 内容生成逻辑: 100%
- TTS集成: > 90%
- 多语言功能: > 85%
- UI组件: > 80%

验收

手工验收清单

- ☐ 内容生成响应时间符合要求(基础<1s, 完整<3s)
- ☐ 6种语言内容生成正常
- ☐ TTS语音合成质量好, 音色可选
- ☐ 流式内容展示流畅, 无卡顿
- ☐ 音频播放控制功能完整
- ☐ 离线模式下可访问已缓存内容
- ☐ 测试覆盖率达标

版本管理和CI/CD

bash

git checkout -b feature/content/ai-explanation

git commit -m "feat(content): implement AI explanation generation"

- Add multi-language content generation with GPT-4
- Integrate TTS service with voice options
- Implement streaming content display
- Add offline content caching
- Achieve 87% test coverage

Closes #3"

Step 3 - 基础UI界面开发

概览

目标

- 设计和实现核心用户界面
- 完善用户交互流程
- 实现响应式设计
- 优化用户体验

Agent角色

优先调用相关agents，如没有匹配的agent你需要扮演以下角色：

1. **UI/UX设计师**: 负责界面设计和用户体验
2. **Flutter UI工程师**: 负责界面实现和动画效果
3. **交互设计师**: 负责用户流程和交互逻辑
4. **可用性测试工程师**: 负责用户体验测试

工作量估算

- 预估Token消耗: 70K tokens
- 预估交互次数: 10-12次
- 预估开发时间: 8-10小时
- 复杂度等级: 中-高

Claude Code完整指令

markdown

GoMuseum架构背景 - UI界面设计

用户界面要求

- **设计风格**: 现代简约, 文化艺术感
- **主色调**: 深蓝 + 金色, 体现博物馆专业性
- **布局**: 底部导航 + 卡片式内容展示
- **动画**: 流畅的转场动画, 识别加载动画
- **响应式**: 支持不同屏幕尺寸和方向

核心页面结构

1. **启动页**: Logo + 加载动画
2. **主页**: 相机识别 + 历史记录 + 个人中心
3. **相机页**: 实时取景 + 拍照按钮 + 识别状态
4. **结果页**: 展品信息 + 讲解内容 + 音频播放
5. **历史页**: 识别历史列表 + 搜索过滤
6. **个人页**: 用户信息 + 设置 + 权益状态

交互流程

主页 → 点击相机 → 拍照识别 → 显示结果 → 播放讲解 → 保存历史

Step 3 - 基础UI界面开发任务

TDD开发模式

重点测试Widget功能和用户交互逻辑。

角色设定

优先调用相关agents, 如没有匹配的agent你需要扮演以下角色:

1. **UI/UX设计师**: 负责界面设计和用户体验
2. **Flutter UI工程师**: 负责界面实现和动画效果
3. **交互设计师**: 负责用户流程和交互逻辑
4. **可用性测试工程师**: 负责用户体验测试

具体开发任务

第一轮TDD - 核心页面开发

****红灯阶段**:**

- 1. 测试主页Widget渲染
- 2. 测试导航功能
- 3. 测试相机页面交互
- 4. 测试结果页面显示

****绿灯阶段**:**

- 1. 实现主页布局和导航
- 2. 实现相机界面和拍照功能
- 3. 实现结果展示页面
- 4. 实现基础路由管理

****重构阶段**:**

- 1. 提取通用UI组件
- 2. 优化布局代码
- 3. 添加响应式支持

第二轮TDD - 交互优化

****红灯阶段**:**

- 1. 测试加载状态显示
- 2. 测试错误状态处理
- 3. 测试手势操作
- 4. 测试动画效果

****绿灯阶段**:**

- 1. 实现加载动画和状态指示
- 2. 实现错误提示和重试机制
- 3. 实现手势识别和操作反馈
- 4. 实现页面转场动画

第三轮TDD - 主题和国际化

****红灯阶段**:**

- 1. 测试主题切换功能
- 2. 测试多语言显示
- 3. 测试可访问性支持
- 4. 测试不同屏幕适配

****绿灯阶段**:**

1. 实现主题管理系统
2. 实现国际化文本支持
3. 实现可访问性标签
4. 实现响应式布局

关键实现文件

UI组件文件

lib/shared/widgets/

├── common/

| ├── app_bar_widget.dart

| ├── bottom_navigation_widget.dart

| ├── loading_widget.dart

| ├── error_widget.dart

| └── empty_state_widget.dart

├── buttons/

| ├── primary_button.dart

| ├── secondary_button.dart

| └── icon_button.dart

├── cards/

| ├── artwork_card.dart

| ├── history_card.dart

| └── info_card.dart

└── animations/

├── fade_transition_widget.dart

├── slide_transition_widget.dart

└── recognition_animation.dart

页面文件

lib/features/

├── home/presentation/pages/

| └── home_page.dart

├── recognition/presentation/pages/

| ├── camera_page.dart

| └── result_page.dart

├── history/presentation/pages/

| └── history_page.dart

└── profile/presentation/pages/

├── profile_page.dart

└── settings_page.dart

主题和国际化

lib/config/

├── theme/

| ├── app_theme.dart

| ├── colors.dart

| ├── text_styles.dart

| └── dimensions.dart

├── localization/

| ├── app_localizations.dart

| └── l10n/

| ├── app_en.arb

| ├── app_zh.arb

| ├── app_fr.arb

| └── app_es.arb

└── routes/

└── app_router.dart

设计规范要求

1. **色彩系统**: 主色#1E3A8A, 辅色#F59E0B, 中性色阶
2. **字体系统**: 标题用Roboto Bold, 正文用Roboto Regular
3. **间距系统**: 8dp网格系统, 标准间距8/16/24/32dp
4. **组件规范**: Material Design 3.0规范
5. **动画**: 250ms标准动画时长, ease-out缓动
6. **暗色模式**: 完整的暗色主题支持

验收标准

1. 所有核心页面UI实现完整
2. 用户交互流程顺畅
3. 响应式设计适配良好
4. 动画效果流畅自然
5. 可访问性支持完善
6. Widget测试覆盖率>75%

请注重用户体验, 确保界面直观易用。

总结

预期输出文件 (35个)

yaml

UI组件文件 (15个):

- 通用组件库 (按钮、卡片、加载等)
- 动画组件
- 布局组件

页面文件 (8个):

- 6个核心页面实现
- 路由配置

配置文件 (7个):

- 主题配置
- 国际化资源文件
- 路由配置

测试文件 (5个):

- Widget测试
- 集成测试

测试覆盖率目标

- Widget测试: > 75%
- 页面交互测试: > 80%
- UI组件测试: > 85%
- 用户流程测试: > 90%

验收

手工验收清单

- ☐ 所有页面UI渲染正常，无布局错误
- ☐ 底部导航工作正常，页面切换流畅
- ☐ 相机界面交互良好，拍照功能正常
- ☐ 结果页面信息展示完整，布局美观
- ☐ 加载和错误状态显示恰当
- ☐ 动画效果流畅，转场自然

- ☐ 多语言切换正常工作
- ☐ 暗色模式支持完整
- ☐ 不同屏幕尺寸适配良好

版本管理和CI/CD

bash

```
git checkout -b feature/ui/core-interface
git commit -m "feat(ui): implement core user interface"
```

- Add complete UI component library
- Implement 6 core pages with Material Design 3.0
- Add theme system with dark mode support
- Implement internationalization for 6 languages
- Add smooth animations and transitions
- Achieve 78% widget test coverage

Closes #4"

Step 4 - 支付功能集成

概览

目标

- 集成Google Play Billing和Apple IAP
- 实现用户权益管理系统
- 支持多种付费模式
- 实现推荐奖励机制

Agent角色

优先调用相关agents，如没有匹配的agent你需要扮演以下角色：

1. **支付系统工程师:** 负责IAP集成和支付流程
2. **权益管理工程师:** 负责用户权益和订阅管理
3. **安全工程师:** 负责支付安全和验证
4. **产品工程师:** 负责付费模式和用户体验

工作量估算

- 预估Token消耗: 40K tokens
- 预估交互次数: 5-7次
- 预估开发时间: 4-6小时
- 复杂度等级: 中

Claude Code完整指令

markdown

GoMuseum架构背景 - 支付系统

商业模式设计

- **免费模式**: 5次免费识别额度
- **按次付费**: €1.99/10次识别
- **按天通行**: €2.99-3.99/天无限识别
- **年度订阅**: €19.9/年会员权益
- **推荐奖励**: 新用户注册+5次, 推荐人获得额外权益

支付技术架构

- **Flutter端**: in_app_purchase插件
- **iOS**: StoreKit 2.0 + 收据验证
- **Android**: Google Play Billing Library 5.0
- **后端验证**: 收据验证 + 订阅状态同步
- **权益管理**: 基于时间和次数的双重控制

安全要求

- 收据验证必须在服务端完成
- 支付状态实时同步
- 防刷单和异常检测
- 用户权益云端备份

Step 4 - 支付功能集成开发任务

TDD开发模式

重点测试支付流程和权益管理逻辑。

角色设定

优先调用相关agents, 如没有匹配的agent你需要扮演以下角色:

1. **支付系统工程师**: 负责IAP集成和支付流程
2. **权益管理工程师**: 负责用户权益和订阅管理
3. **安全工程师**: 负责支付安全和验证
4. **产品工程师**: 负责付费模式和用户体验

具体开发任务

第一轮TDD - 基础支付集成

红灯阶段:

1. 测试商品信息获取
2. 测试支付流程启动
3. 测试支付成功回调
4. 测试支付失败处理

绿灯阶段:

1. 集成in_app_purchase插件
2. 配置iOS和Android商品
3. 实现支付流程管理
4. 实现支付状态处理

重构阶段:

1. 抽象支付接口
2. 优化错误处理
3. 完善日志记录

第二轮TDD - 权益管理系统

红灯阶段:

1. 测试用户权益查询
2. 测试权益消费逻辑
3. 测试权益同步机制
4. 测试订阅状态管理

绿灯阶段:

1. 实现权益数据模型
2. 实现权益消费控制
3. 实现云端权益同步
4. 实现订阅自动续费

第三轮TDD - 推荐奖励系统

红灯阶段:

1. 测试推荐码生成
2. 测试推荐关系绑定
3. 测试奖励发放逻辑

4. 测试防作弊机制

****绿灯阶段**:**

1. 实现推荐码系统
2. 实现奖励计算逻辑
3. 实现奖励发放机制
4. 实现反作弊检测

关键实现文件

Flutter端文件

lib/features/payment/

```
├── data/
|   ├── datasources/
|   |   ├── iap_datasource.dart
|   |   └── benefits_remote_datasource.dart
|   └── models/
|       ├── product_model.dart
|       ├── purchase_model.dart
|       └── benefits_model.dart
|   └── repositories/
|       └── payment_repository_impl.dart
├── domain/
|   ├── entities/
|   |   ├── product.dart
|   |   ├── purchase.dart
|   |   └── user_benefits.dart
|   └── repositories/
|       └── payment_repository.dart
└── usecases/
    ├── purchase_product.dart
    └── restore_purchases.dart
```

```
|   |—— check_benefits.dart
|   |—— redeem_referral_code.dart
|   |—— presentation/
|   |—— providers/
|   |—— payment_provider.dart
|   |—— benefits_provider.dart
|   |—— pages/
|   |—— purchase_page.dart
|   |—— benefits_page.dart
|   |—— referral_page.dart
|   |—— widgets/
|   |—— product_card.dart
|   |—— benefits_status.dart
|   |—— referral_code_widget.dart
```

后端文件

```
backend/app/
|   |—— api/v1/
|   |—— payment.py
|   |—— benefits.py
|   |—— referral.py
|   |—— services/
|   |—— iap_verification_service.py
|   |—— benefits_service.py
|   |—— subscription_service.py
|   |—— referral_service.py
|   |—— models/
|   |—— purchase.py
|   |—— subscription.py
|   |—— benefits.py
```

| └── referral.py
└── schemas/
├── purchase_request.py
├── benefits_response.py
└── referral_request.py

商品配置

```yaml

产品定义:

com.gomuseum.recognition\_pack\_10:

name: "10次识别包"

price: "€1.99"

type: "consumable"

com.gomuseum.day\_pass:

name: "一日通行证"

price: "€2.99"

type: "non\_consumable"

com.gomuseum.premium\_annual:

name: "年度会员"

price: "€19.90"

type: "auto\_renewable\_subscription"

## 验收标准

1. iOS和Android支付流程正常
2. 收据验证机制安全可靠
3. 用户权益管理准确无误
4. 推荐奖励系统工作正常
5. 支付异常处理完善
6. 测试覆盖率>85%

请确保支付安全，所有收据验证必须在服务端完成。

### 总结

#### 预期输出文件 (18个)

``yaml

Flutter代码文件 (10个):

- 支付相关的完整Clean Architecture实现
- IAP集成和权益管理
- 推荐系统UI组件

Python后端文件 (6个):

- 支付验证和权益管理API
- 推荐奖励系统
- 订阅管理服务

测试文件 (2个):

- 支付流程测试
- 权益管理测试

测试覆盖率目标

- 支付流程: > 90%
- 权益管理: > 95%
- 推荐系统: > 85%

验收

手工验收清单

- ☐ iOS App Store支付流程正常
- ☐ Android Google Play支付流程正常
- ☐ 收据验证安全可靠
- ☐ 用户权益显示准确
- ☐ 推荐码生成和使用正常

- ☐ 订阅自动续费工作
- ☐ 支付异常处理恰当

## 版本管理和CI/CD

bash

```
git checkout -b feature/payment/iap-integration
git commit -m "feat(payment): implement in-app purchase system"
```

- Integrate iOS StoreKit and Android Play Billing
- Add user benefits management system
- Implement referral reward mechanism
- Add secure receipt verification
- Achieve 88% payment test coverage

Closes #5"

## Step 5 - 测试和优化

### 概览

### 目标

- 完善整体测试覆盖率
- 进行性能优化和压力测试
- 修复发现的Bug和问题
- 准备MVP版本发布

### Agent角色

优先调用相关agents，如没有匹配的agent你需要扮演以下角色：

1. QA测试工程师: 负责全面测试和质量保证
2. 性能优化工程师: 负责性能分析和优化

3. **DevOps工程师:** 负责发布准备和部署

4. **产品验收工程师:** 负责最终产品验收

工作量估算

- 预估Token消耗: 40K tokens
- 预估交互次数: 5-7次
- 预估开发时间: 4-6小时
- 复杂度等级: 中

Claude Code完整指令

markdown

## # GoMuseum架构背景 - 测试和优化

### ## 性能目标

- 识别响应时间: P95 < 5秒
- 缓存命中率: > 70%
- 应用启动时间: < 3秒
- 内存使用: < 200MB
- 包大小: < 50MB

### ## 测试覆盖率要求

- Flutter单元测试: > 80%
- Flutter Widget测试: > 75%
- Python单元测试: > 85%
- 集成测试: > 80%
- E2E测试: 覆盖主要用户路径

### ## 质量指标

- Crash率: < 0.1%
- ANR率: < 0.5%
- 用户体验评分: > 4.0
- 性能评分: > 80

---

## # Step 5 - 测试和优化任务

### ## TDD开发模式

重点完善测试覆盖率和性能优化。

### ## 角色设定

优先调用相关agents, 如没有匹配的agent你需要扮演以下角色:

1. **QA测试工程师**: 负责全面测试和质量保证
2. **性能优化工程师**: 负责性能分析和优化
3. **DevOps工程师**: 负责发布准备和部署
4. **产品验收工程师**: 负责最终产品验收

### ## 具体开发任务

### ### 第一轮 - 测试覆盖率完善

#### \*\*任务内容\*\*:

1. 补充缺失的单元测试
2. 增加Widget测试覆盖
3. 完善集成测试
4. 添加E2E测试用例

#### \*\*测试重点\*\*:

1. 核心业务逻辑测试
2. 边界条件和异常处理
3. 用户交互流程测试
4. 性能和压力测试

### ### 第二轮 - 性能优化

#### \*\*优化重点\*\*:

1. 识别响应时间优化
2. 内存使用优化
3. 网络请求优化
4. UI渲染性能优化

#### \*\*具体措施\*\*:

1. 实现智能缓存策略
2. 优化图像处理流程
3. 减少不必要的重建
4. 异步加载优化

### ### 第三轮 - Bug修复和发布准备

#### \*\*修复重点\*\*:

1. 修复测试发现的Bug
2. 完善错误处理机制
3. 优化用户体验细节
4. 准备发布配置

### ## 关键测试文件

### ### 完整测试套件

```
test/
├── unit/
│ ├── features/
│ │ ├── recognition/
│ │ │ ├── data/
│ │ │ ├── domain/
│ │ │ └── presentation/
│ │ ├── content/
│ │ ├── payment/
│ │ └── auth/
│ ├── core/
│ │ ├── network/
│ │ ├── storage/
│ │ └── utils/
│ └── shared/
├── widget/
│ ├── pages/
│ ├── widgets/
│ └── flows/
├── integration/
│ ├── recognition_flow_test.dart
│ ├── payment_flow_test.dart
│ └── content_generation_test.dart
└── e2e/
 ├── complete_user_journey_test.dart
 ├── offline_mode_test.dart
 └── payment_process_test.dart
```

### 性能测试文件

test/performance/  
├── recognition\_performance\_test.dart  
├── ui\_performance\_test.dart  
├── memory\_usage\_test.dart  
└── network\_performance\_test.dart

### 后端测试

backend/tests/  
├── unit/  
│ ├── test\_recognition\_api.py  
│ ├── test\_content\_generation.py  
│ ├── test\_payment\_verification.py  
│ └── test\_benefits\_management.py  
├── integration/  
│ ├── test\_ai\_service\_integration.py  
│ ├── test\_database\_operations.py  
│ └── test\_cache\_operations.py  
├── e2e/  
│ ├── test\_complete\_recognition\_flow.py  
│ ├── test\_payment\_flow.py  
│ └── test\_content\_generation\_flow.py  
└── performance/  
 ├── test\_api\_performance.py  
 ├── test\_concurrent\_requests.py  
 └── test\_cache\_performance.py

## ## 性能优化清单

### 1. \*\*识别优化\*\*:

- 实现图像预处理优化
- 添加本地特征缓存
- 优化API调用策略

### 2. \*\*UI优化\*\*:

- 减少Widget重建
- 实现图片懒加载
- 优化动画性能

### 3. \*\*网络优化\*\*:

- 实现请求去重
- 添加网络缓存
- 优化超时策略

### 4. \*\*内存优化\*\*:

- 及时释放资源
- 优化图片内存占用
- 减少内存泄漏

## ## 发布准备清单

### 1. \*\*代码质量\*\*:

- 通过所有静态分析
- 测试覆盖率达标
- 性能指标达标

### 2. \*\*配置准备\*\*:

- 生产环境配置
- API密钥配置
- 应用签名配置

### 3. \*\*文档准备\*\*:

- 用户使用说明
- 开发文档更新
- 发布说明文档

## ## 验收标准

1. 所有测试用例通过
2. 性能指标达到目标
3. 用户体验流畅
4. 无严重Bug和崩溃
5. 发布配置正确
6. 文档完整

请确保质量达到发布标准。

## 总结

### 预期输出文件 (25个)

yaml

#### 测试文件 (20个):

- 完整的单元测试套件
- Widget和集成测试
- E2E端到端测试
- 性能测试

#### 优化代码 (3个):

- 性能优化相关修改
- 缓存策略优化
- UI性能优化

#### 配置文件 (2个):

- 生产环境配置
- 发布配置文件

### 最终测试覆盖率目标

- Flutter单元测试: > 80%
- Flutter Widget测试: > 75%

- Python单元测试: > 85%
- 集成测试: > 80%
- E2E测试: 100%覆盖主要流程

## 验收

### 最终验收清单

- ☐ 识别功能稳定，响应时间达标
- ☐ AI讲解生成正常，内容质量良好
- ☐ UI界面美观流畅，用户体验佳
- ☐ 支付功能正常，权益管理准确
- ☐ 测试覆盖率全部达标
- ☐ 性能指标全部达标
- ☐ 无严重Bug和崩溃
- ☐ 发布配置准备完成

### 性能验收脚本

```
bash
```

```
#!/bin/bash
```

```
mvp-final-acceptance.sh
```

```
echo "MVP最终验收测试"
```

```
运行所有测试
```

```
flutter test --coverage
```

```
pytest backend/tests/ --cov=app --cov-report=term-missing
```

```
性能测试
```

```
flutter drive --target=test_driver/performance_test.dart
```

```
检查性能指标
```

```
echo "检查性能指标..."
```

```
响应时间、内存使用、包大小等检查
```

```
echo "MVP验收完成"
```

## 版本管理和CI/CD

### 最终发布流程

```
bash
```

# 合并所有功能分支到staging

git checkout staging

git merge feature/recognition/core-implementation

git merge feature/content/ai-explanation

git merge feature/ui/core-interface

git merge feature/payment/iap-integration

# 最终测试和优化

git checkout -b release/mvp-v1.0.0

git commit -m "release: prepare MVP v1.0.0"

- Complete recognition and explanation features
- Implement full UI and payment system
- Achieve target test coverage and performance
- Ready for production deployment

Features:

-  5-second image recognition
-  Multi-language AI explanations
-  In-app purchase system
-  Comprehensive user interface
-  80%+ test coverage

Performance:

-  P95 response time < 5s
-  70%+ cache hit rate
-  <200MB memory usage

Release #MVP-1.0"

# 合并到main进行生产部署

git checkout main

git merge release/mvp-v1.0.0

git tag v1.0.0

# MVP开发总结

## 完整功能清单

1.  图像识别: 5秒内识别展品, 支持缓存
2.  AI讲解: 多语言内容生成, TTS语音合成
3.  用户界面: 完整的UI/UX, 6种语言支持
4.  支付系统: IAP集成, 权益管理, 推荐奖励
5.  质量保证: 全面测试, 性能优化

## 技术成果

- 代码文件: 100+ 个核心文件
- 测试覆盖率: Flutter 80%+, Python 85%+
- 性能指标: 全部达标
- 代码质量: 通过所有静态分析

## 下一步计划

MVP完成后, 可以进入后续的功能增强阶段:

- **Step 6:** 离线包功能
- **Step 7:** 性能优化和缓存增强
- **Step 8:** 用户体验优化
- **Step 9:** 数据分析和推荐系统
- **Step 10:** 规模化部署和运维

总Token消耗预估: 280K tokens (5个Step) 总开发时间预估: 30-40小时 发布时间:  
MVP开发完成后2-3周内发布